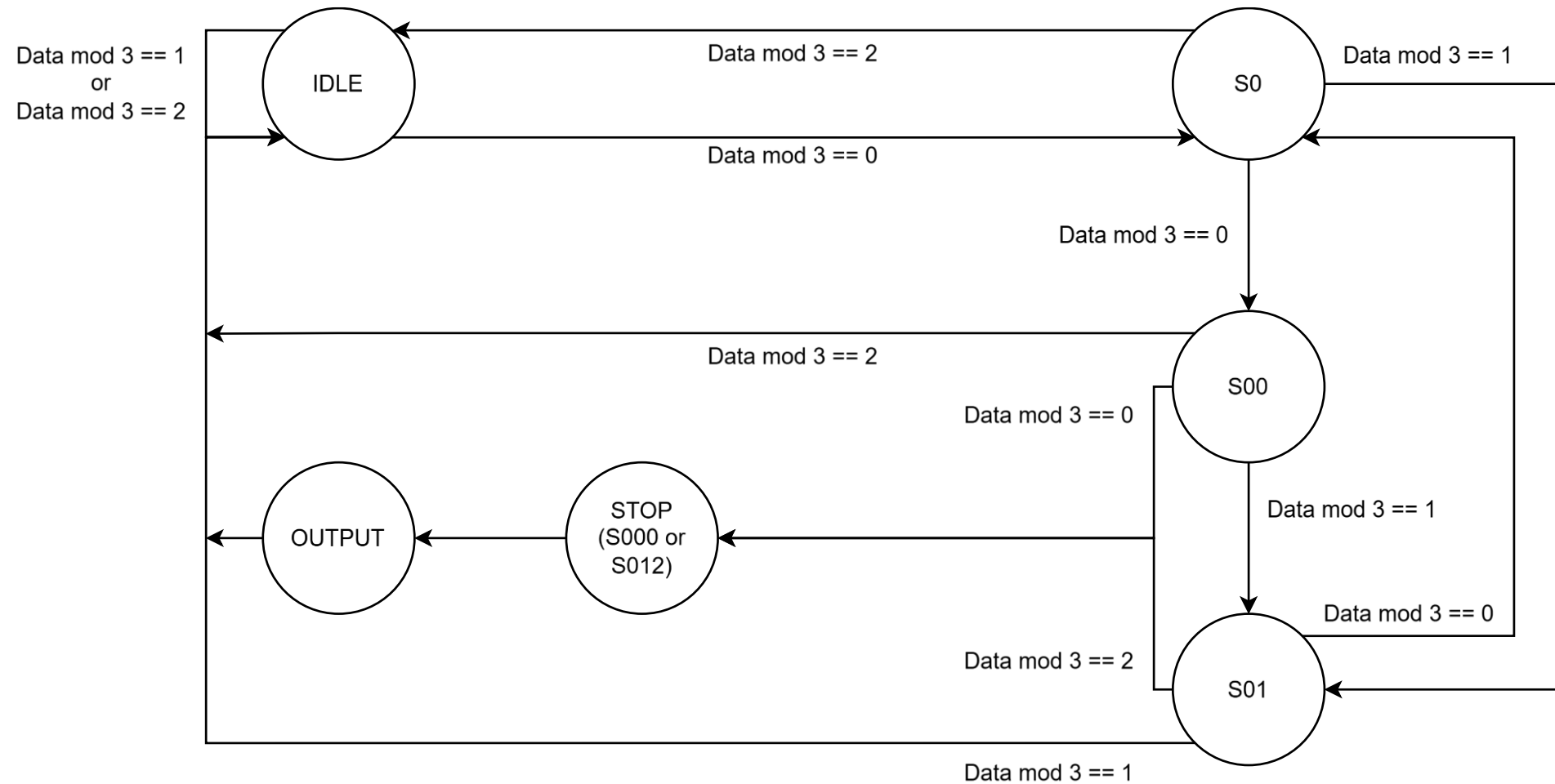




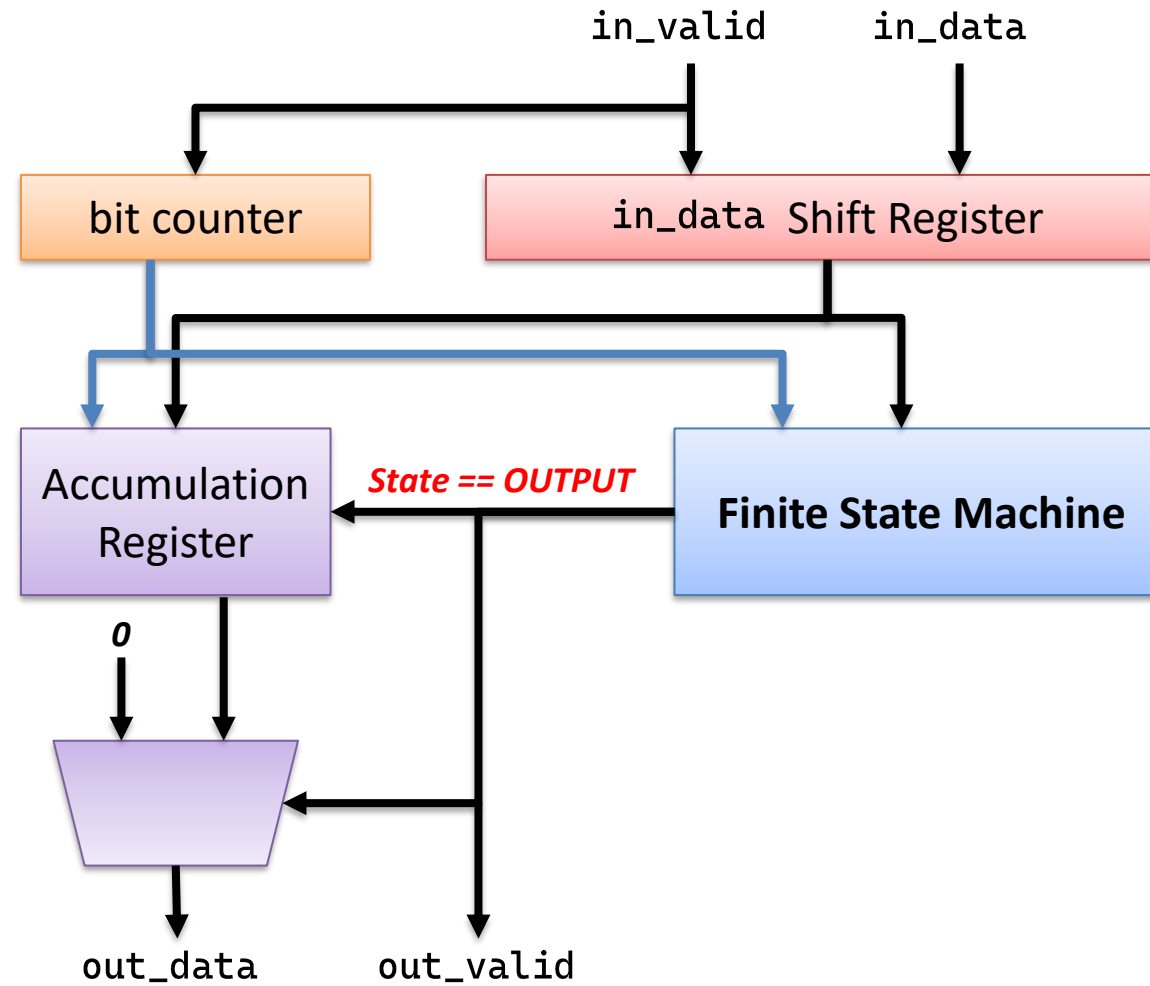
Lab04 Code Review

許誠

State Diagram

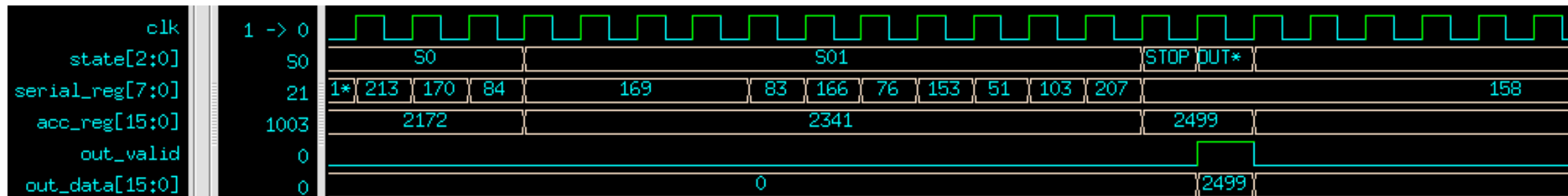
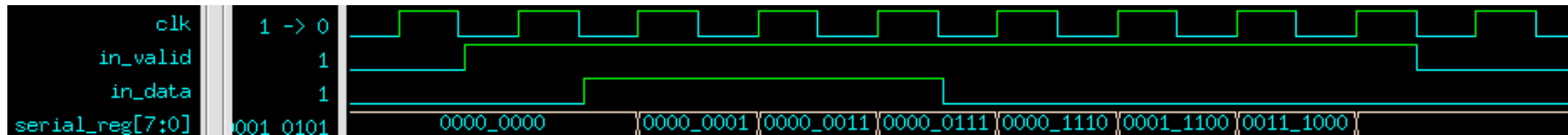


Block Diagram



Timing diagram for the bit counter module. The diagram shows the following signals and their values over time:

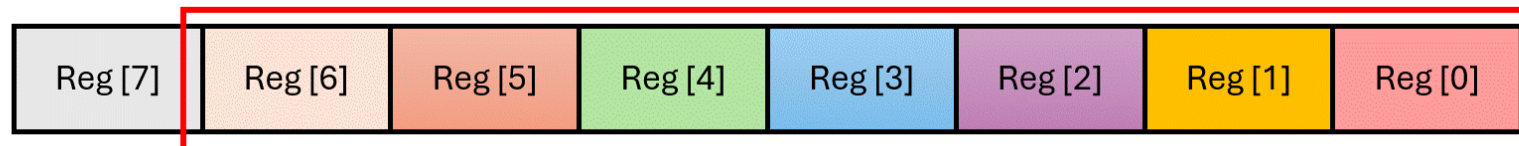
- state[2:0]**: S0
- clk**: 1 (clock signal)
- in_valid**: 1 (active low signal, pulses at the start of each 8-bit data cycle)
- bit_cnt[2:0]**: 2 (bit index, values 0, 1, 2, 3, 4, 5, 6, 7)
- acc_reg[15:0]**: 1003 (accumulated value, values 0, 112, 338)



Reference Code (Shift register)

- Shift register for in_data

```
always_ff @( posedge clk or negedge rst_n ) begin
    if ( !rst_n ) begin
        serial_reg <= 0;
    end else begin
        if(in_valid) begin
            serial_reg <= {serial_reg[6:0], in_data};
        end
    end
end
end
```



Reference Code (FSM)

- if (bit_cnt == 7)
- 每收到8個 in_data 判斷一次 next_state

```

always_comb begin
    next_state = state;
    case(state)
        IDLE: begin
            if (bit_cnt == 7) begin
                if ({serial_reg[6:0], in_data} % 3 == 0) begin
                    next_state = S0;
                end else begin
                    next_state = IDLE;
                end
            end
        end
        S0: begin
            if (bit_cnt == 7) begin
                if ({serial_reg[6:0], in_data} % 3 == 0) begin
                    next_state = S00;
                end else if ({serial_reg[6:0], in_data} % 3 == 1) begin
                    next_state = S01;
                end else begin
                    next_state = IDLE;
                end
            end
        end
        S00: begin
            if (bit_cnt == 7) begin
                if ({serial_reg[6:0], in_data} % 3 == 0) begin
                    next_state = STOP;
                end else if ({serial_reg[6:0], in_data} % 3 == 1) begin
                    next_state = S01;
                end else begin
                    next_state = IDLE;
                end
            end
        end
        S01: begin
            if (bit_cnt == 7) begin
                if ({serial_reg[6:0], in_data} % 3 == 2) begin
                    next_state = STOP;
                end else if ({serial_reg[6:0], in_data} % 3 == 0) begin
                    next_state = S0;
                end else begin
                    next_state = IDLE;
                end
            end
        end
        STOP: begin
            next_state = OUTPUT;
        end
        OUTPUT: begin
            next_state = IDLE;
        end
    endcase
end

```

Coding style – Flip-flop & Reset Signal



```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n || state != out) begin
        out_valid <= 0;
        out_data <= 0;
    end
    else if(state == out) begin
        out_valid <= 1;
        out_data <= sum;
    end
end
```



```
elaborate $DESIGN
Loading db file '/usr/cad/synopsys/synthesis/2022.03/libraries/syn/gtech.db'
Loading link library 'slow'
Loading link library 'gtech'
Running PRESTO HDLC
Error: ../01_RTL/DCSPI.sv:51: The expression in the reset condition of the 'if' statement in this 'always' block can only be
Error: ../01_RTL/DCSPI.sv:55: The expression in the reset condition of the 'if' statement in this 'always' block can only be
*** Presto compilation terminated with 2 errors. ***
0
# (B-2) read_sverilog
# read_sverilog $DESIGN.v
# (B-3) set current design
current_design $DESIGN
Error: Can't find design 'DCSPI'. (UID-109)
Error: Current design is not defined. (UID-4)
link
Error: Current design is not defined. (UID-4)
0
```

Do not write other logic in the reset expression !!

Coding style – Flip-flop & Reset Signal



```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        out_valid <= 0;
        out_data <= 0;
    end else begin
        if(state == out) begin
            out_valid <= 1;
            out_data <= sum;
        end else begin
            out_valid <= 0;
            out_data <= 0;
        end
    end
end
end
```